

Syntax Directed Translation in Compiler Design

Syntax Directed Definition

SDD = $\begin{matrix} \text{CFG} \\ | \\ \text{Context Free Grammar} \end{matrix} + \text{Semantic Rules}$

→ A SDD is a Context Free grammar together with Semantic Rules

→ Attributes are associated with grammar symbols and Semantic Rules are associated with productions.

→ If 'x' is a symbol and 'a' is one of its attribute then $x.a$ denotes value of node 'x'.

→ Attributes may be numbers, strings, references, datatypes etc.

production

Semantic Rule

$E \rightarrow E + T$

$E.val = E.val + T.val$

$E \rightarrow T$

$E.val = T.val$

Types of Attributes:-

1. Synthesized Attribute :- If a node takes value from its children then its synthesized attribute.

Ex:- $A \rightarrow BCD$, A be a parent node
B, C, D are children nodes.

$$\left. \begin{array}{l} A.S = B.S \\ A.S = C.S \\ A.S = D.S \end{array} \right\} \begin{array}{l} \text{parent node A taking value} \\ \text{from its children B, C, D} \end{array}$$

2. Inherited Attribute :- If a node takes value from its parent or siblings.

Ex:- $A \rightarrow BCD$

$$\begin{array}{ll} C.i = A.i & \rightarrow \text{parent node} \\ C.i = B.i & \rightarrow \text{sibling node} \\ C.i = D.i & \rightarrow \text{sibling node} \end{array}$$

Types of SDD:-

1. S-Attributed SDD (or) S-Attributed Definitions (or) S-Attributed grammar
2. L-Attributed SDD (or) L-Attributed Definitions (or) L-Attributed grammar

S-Attributed SDD

1. A SDD that uses only synthesized attributes is called as S-Attributed SDD.

Ex:- $A \rightarrow BCD$

$$A.S = B.S$$

$$A.S = C.S$$

$$A.S = D.S$$

2. Semantic Actions are always placed at right end of the production. It is also called as "postfix SDD".

3. Attributes are evaluated with Bottom-up parsing.

L-Attributed SDD

1. A SDD that uses both synthesized & inherited attributes is called L-Attributed SDD but each inherited attribute is restricted to inherit from parent or left sibling only.

Ex:- $A \rightarrow XYZ \{ \begin{matrix} x.s = A.s \\ y.s = x.s \end{matrix} \}$

2. Semantic Actions are placed anywhere on RHS.

3. Attributes are evaluated by traversing parse tree depth first, left to right order.

Top down
right
left to
right

$x.s = 2.s$

Creating Parse Tree

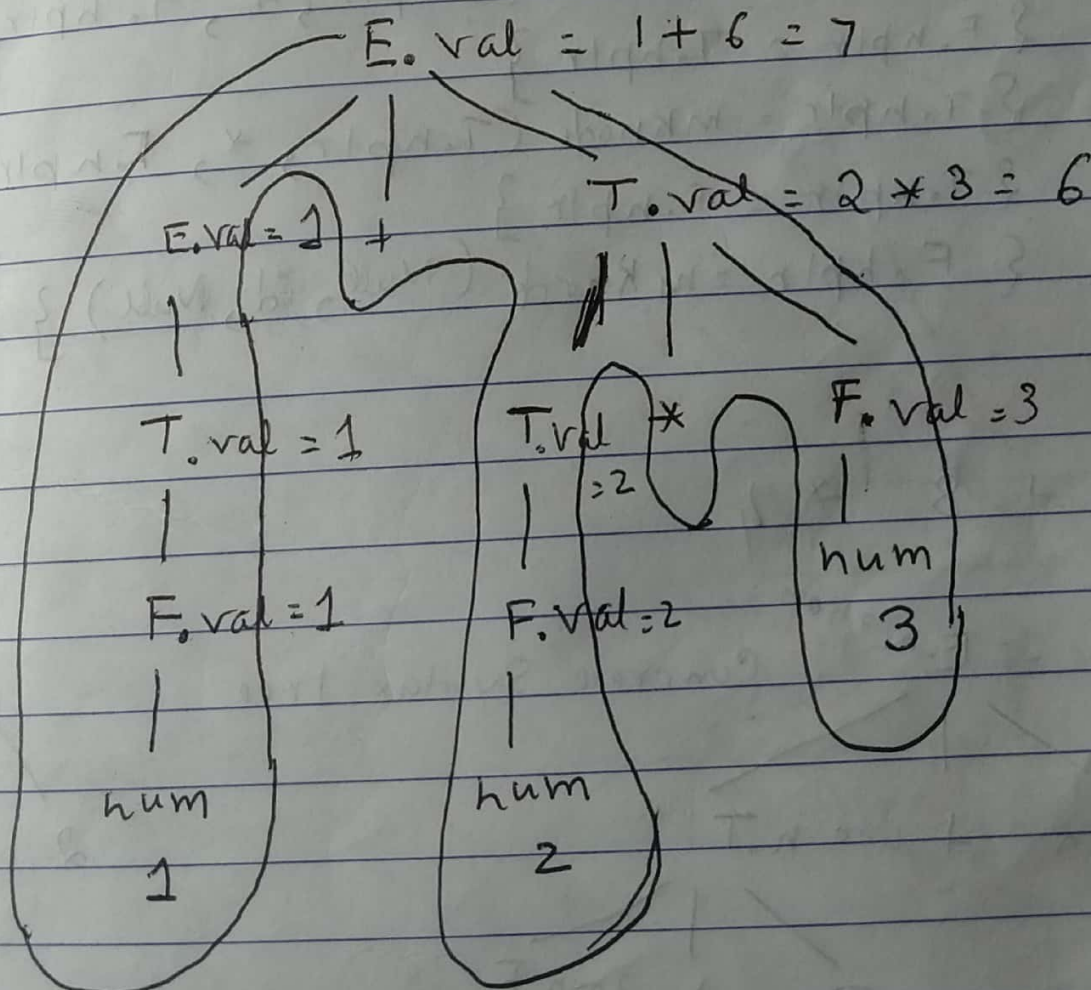
Grammar

Actions { Semantic Rules }

$E \rightarrow E + T$	$\{ E.val = E.val + T.val \}$
$E \rightarrow T$	$\{ E.val = T.val \}$
$T \rightarrow T * F$	$\{ T.val = T.val * F.val \}$
$T \rightarrow F$	$\{ T.val = F.val \}$
$F \rightarrow num$	$\{ F.val = num.lval \}$

Eg:

$1 + 2 * 3$



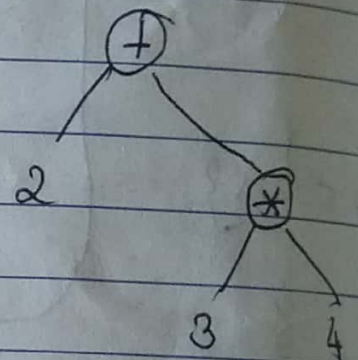
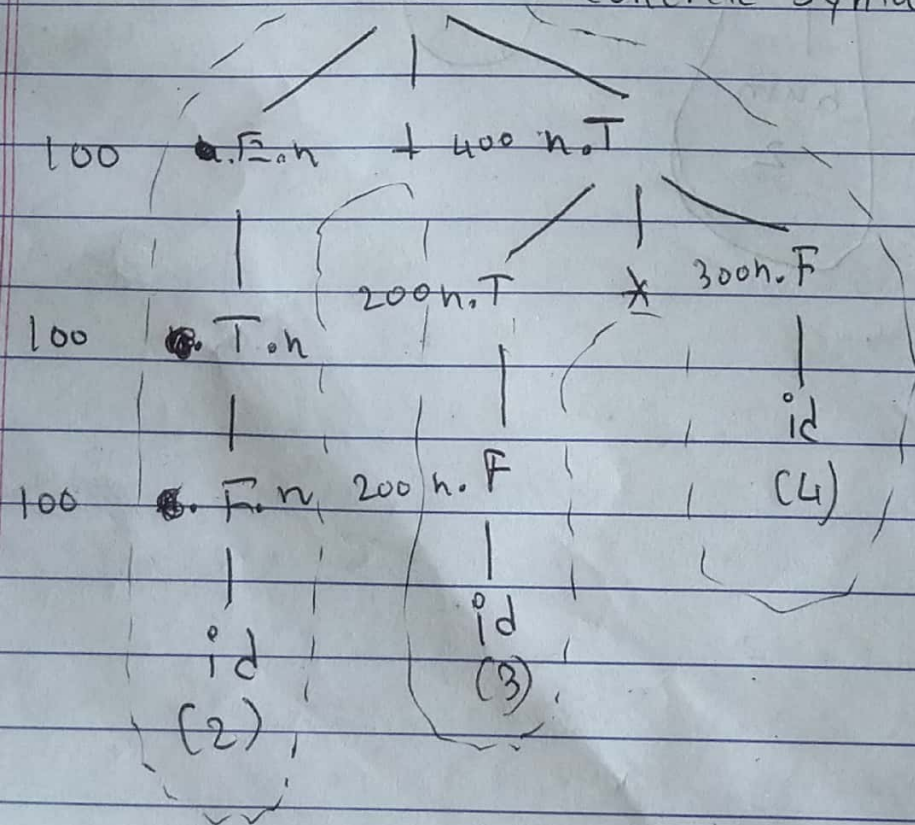
A notated parse tree

$E \rightarrow E + T \quad \{ E.hptr = \text{mknode}(F.hptr, +, T.hptr) \}$
 $E \rightarrow T \quad \{ E.hptr = T.hptr \}$
 $T \rightarrow T * F \quad \{ T.hptr = \text{mknode}(T.hptr, *, F.hptr) \}$
 $T \rightarrow F \quad \{ T.hptr = F.hptr \}$
 $F \rightarrow id \quad \{ F.hptr = \text{mknode}(\text{Null}, id, \text{Null}) \}$

2 + 3 * 4

E.h²⁵⁰⁰

Concrete Syntax tree



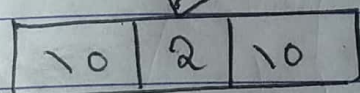
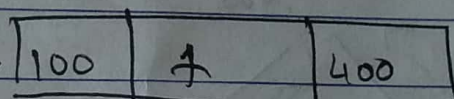
Abstract Syntax tree

T → F

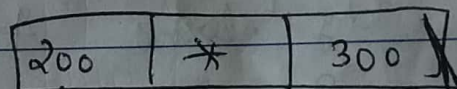
F → id

100

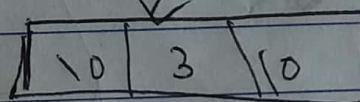
Null id Null



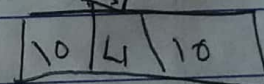
F.ptr



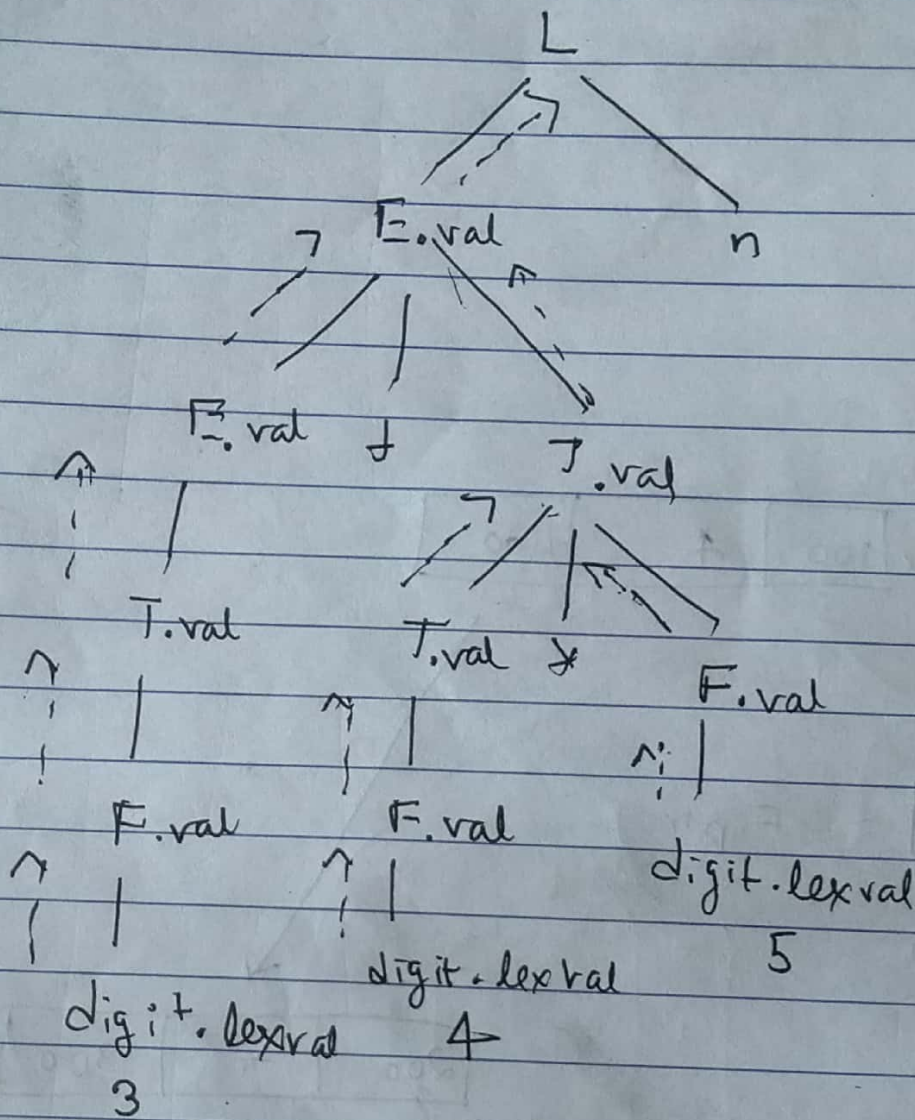
400



200



300



Stack with 2 array

Symbol	Attributes	Value
E		

Input
3 + 4 * 5

A → XYZ

RHS is reduced to LHS
& updated accordingly

P/p

Stack

Reducing

Production Used

$L \rightarrow E_n$

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

$3 + 4 * 5n$

$+ 4 * 5n$

~~top~~

1	1
3	-

$+ 4 * 5n$

F	3

$F \rightarrow \text{digit}$

$+ 4 * 5n$

T	3
---	---

$T \rightarrow F$

$+ 4 * 5n$

E	3
---	---

$E \rightarrow T$

$4 * 5n$

+	-
E	3

$* 5n$

4	-
+	-
E	3

$* 5n$

F	4
+	-
E	3

$F \rightarrow \text{digit}$

$* 5n$

T	4
+	-
E	3

$T \rightarrow F$

$5n$

*	-
T	4
+	-
E	3

i/p

Stack

Production Used

h

5	-
*	-
T	4
+	-
E	3

7

h

F	5
*	-
T	4
+	-
E	3

$F \Rightarrow \text{digit}$

h

T	20
+	-
E	3

$T \rightarrow T * F$

n

	1
E	23

$$E \rightarrow E + T$$

-

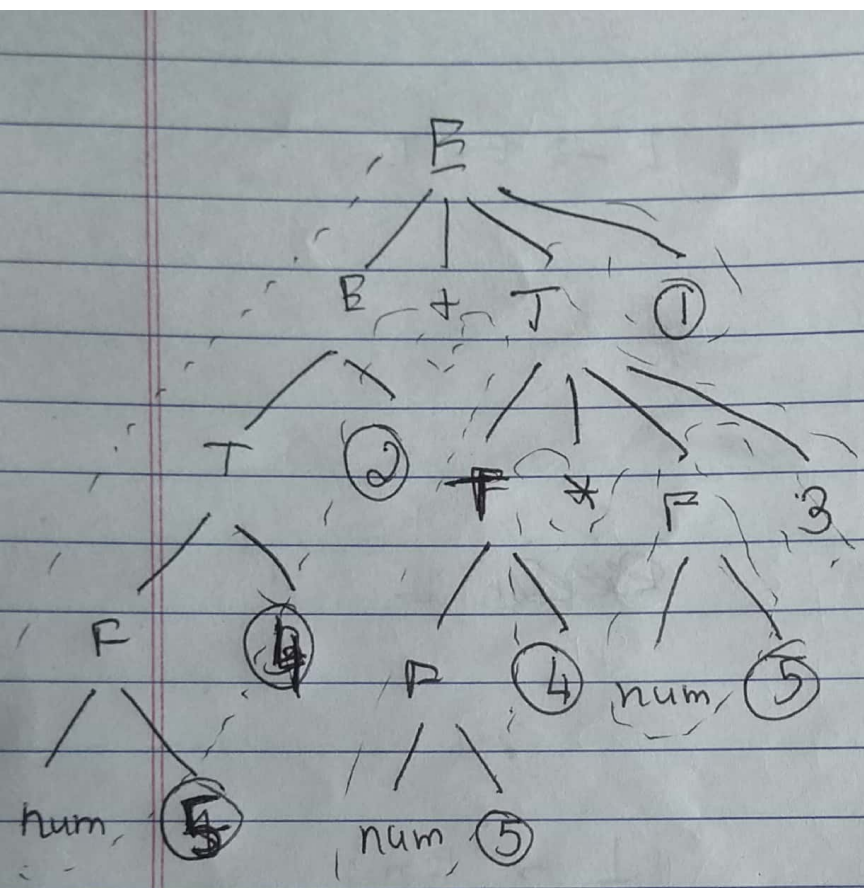
n	-
E	23

$$E \rightarrow E + n$$

-

L	23

$$L \rightarrow E + n$$



$E \rightarrow E + T$ {1}
 $E \rightarrow T$ {2}
 $T \rightarrow T * F$ {3}
 $T \rightarrow F$ {4}
 $F \rightarrow num$ {5}

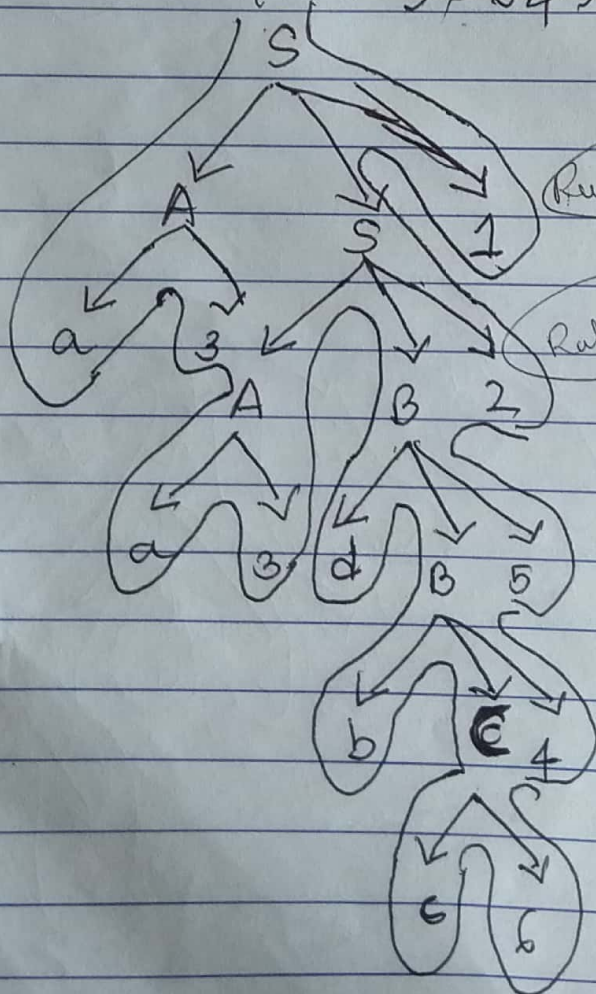
i/p num + num * num
 op ~~5 4 3 2 1~~

Syntax Directed Translations

Top down

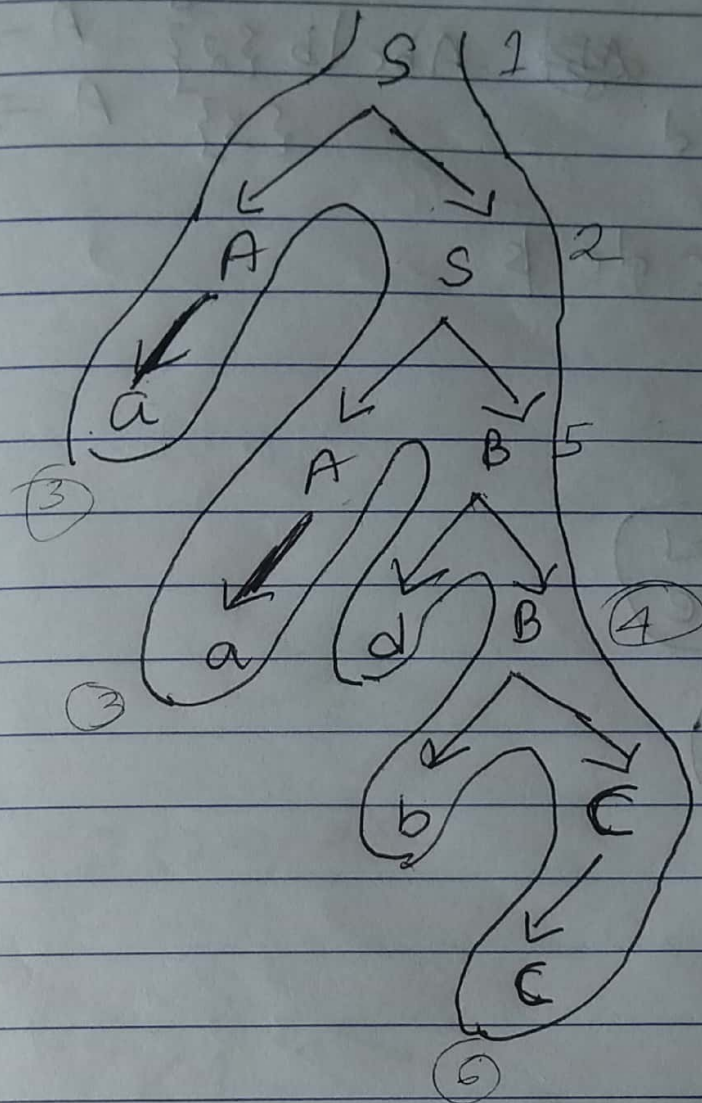
$S \rightarrow AS$	$\{ \text{printf} ("1"); \}$
$S \rightarrow AB$	$\{ \text{printf} ("2"); \}$
$A \rightarrow a$	$\{ \text{printf} ("3"); \}$
$B \rightarrow b$	$\{ \text{printf} ("4"); \}$
$B \rightarrow dB$	$\{ \text{printf} ("5"); \}$
$C \rightarrow c$	$\{ \text{printf} ("6"); \}$

I/p = aadbC
O/p 3364521



Top to down

Left to down



Reduce

reduction

~~3 3 6 4 5 2 1~~ ~~3 3~~

3 3 6 4 5 2 1

$S \rightarrow aABe$ $\{1\}$

$A \rightarrow \cancel{Ab} Abc | b$ $\{2\}$

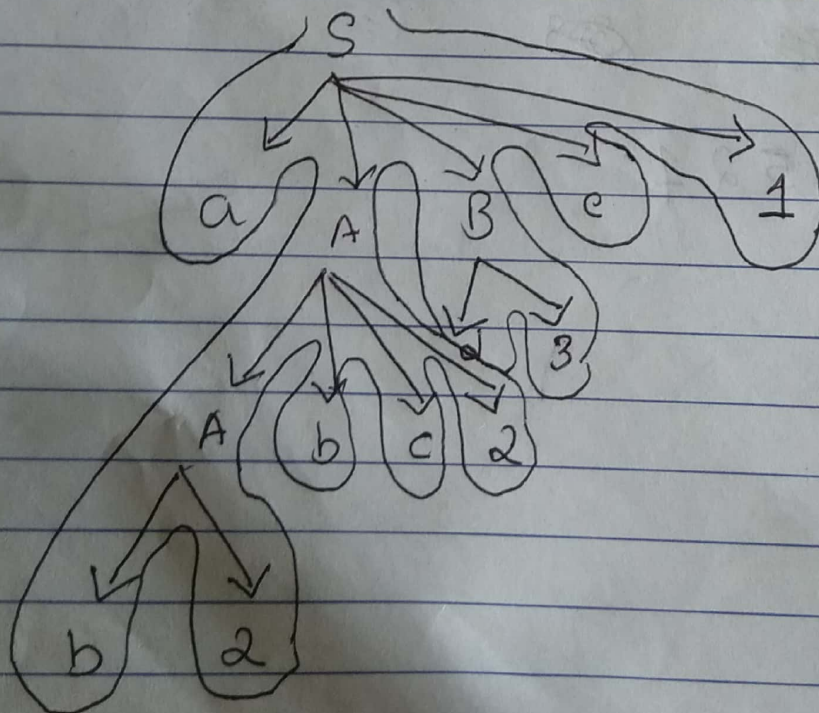
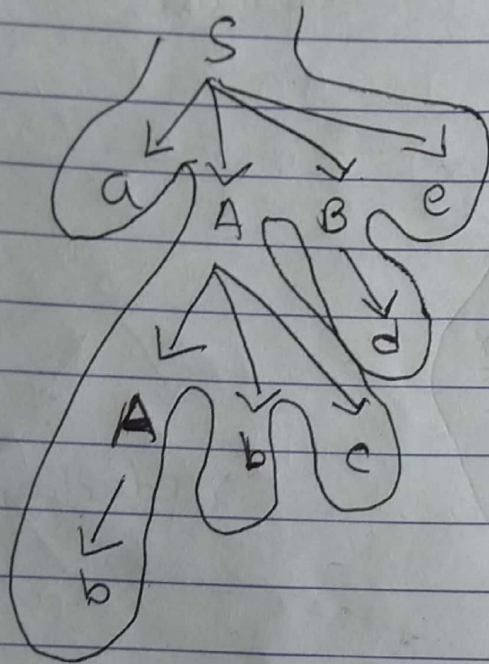
$B \rightarrow d$

$\{3\}$

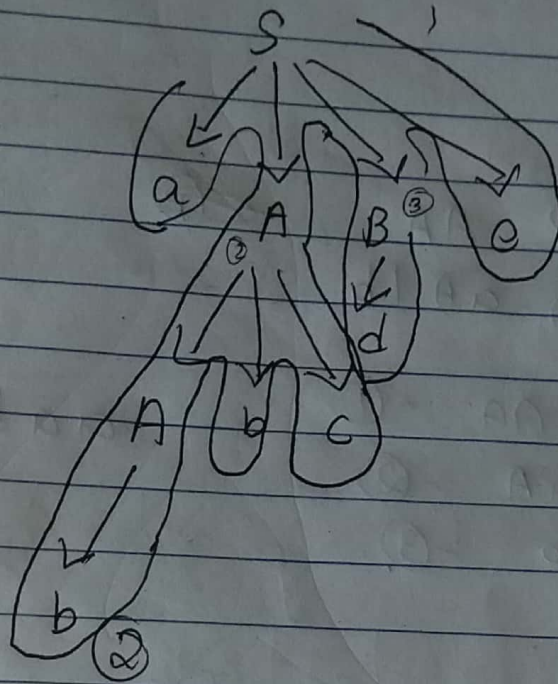
$A \rightarrow Abc$

$A \rightarrow b$

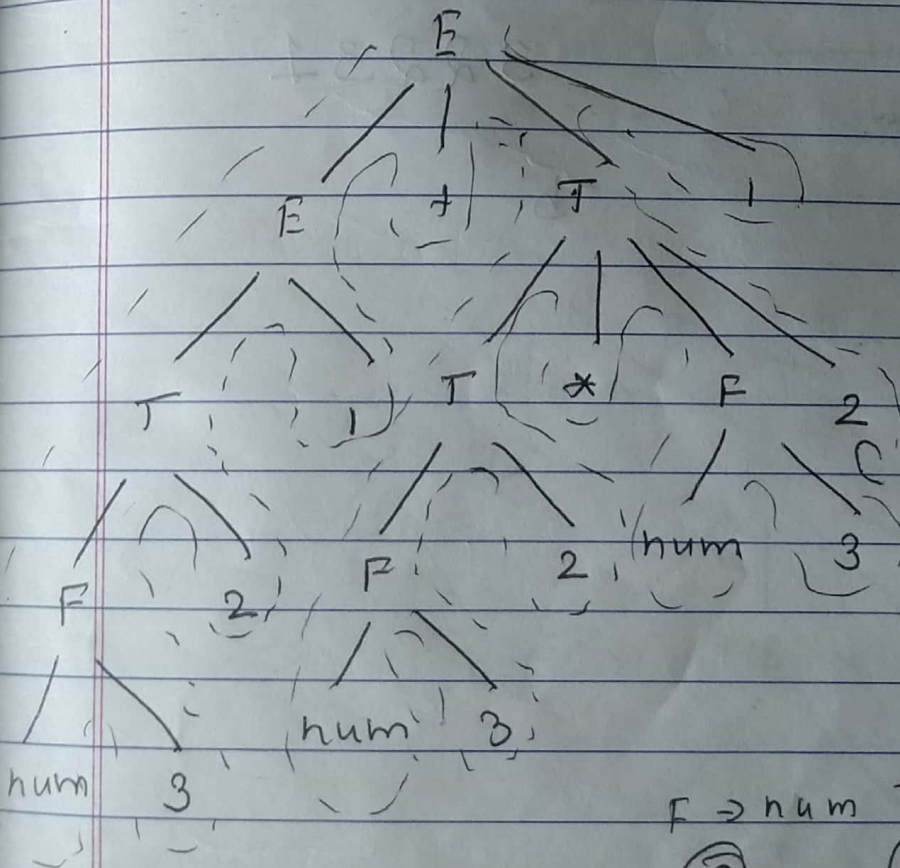
$a b b c d e \mid$



2 2 3 1



2 2 3 1



type num + num * num
o/p: 3 2 1 3 2 3 2 1

$F \rightarrow \text{num}$ $T \rightarrow F$ $E \rightarrow T$ $F \rightarrow \text{num}$
 (3) (2) (1) (3)

$T \rightarrow F$ $F \rightarrow \text{num}$ $T \rightarrow T \wedge F$ $E \rightarrow E + T$
 (2) (3) (2) (1)

$S \rightarrow AA$ ~~(1)~~

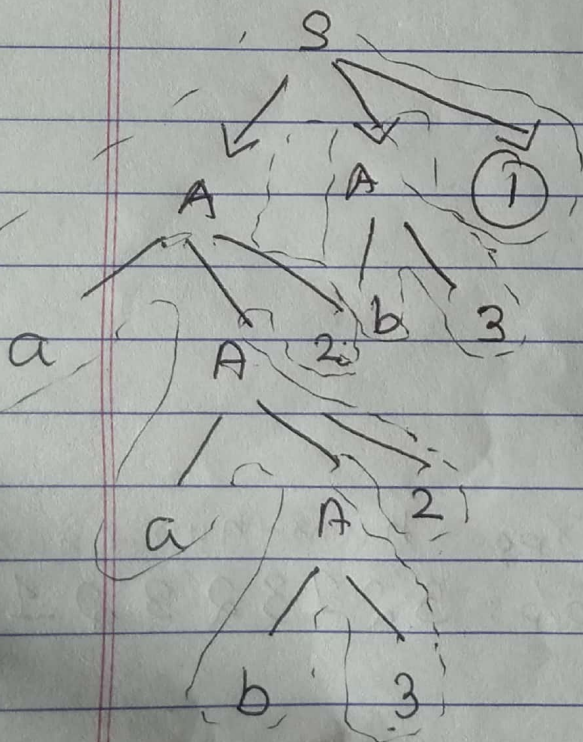
$A \rightarrow aAb$

$S \rightarrow AA$ - (1)

$A \rightarrow aA$ - (2)

$A \rightarrow b$ - (3)

$aabb$



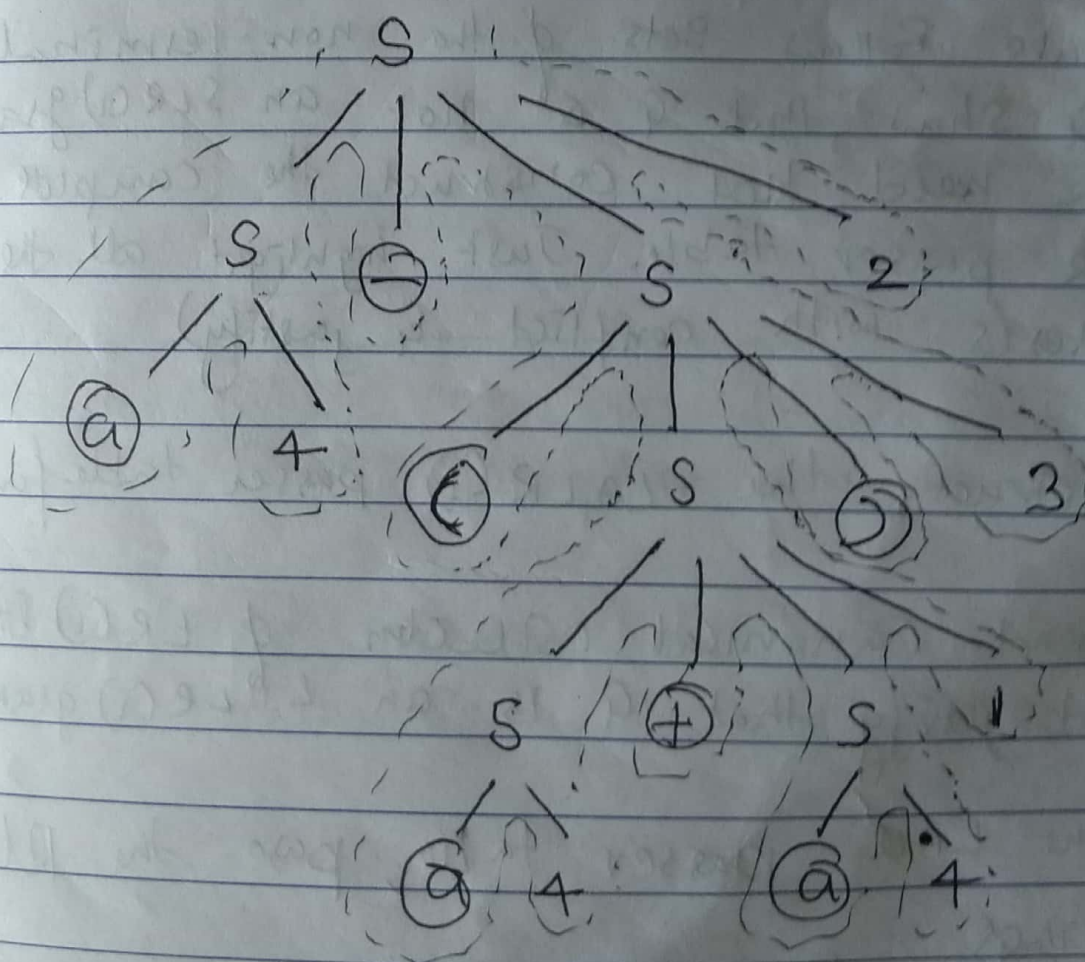
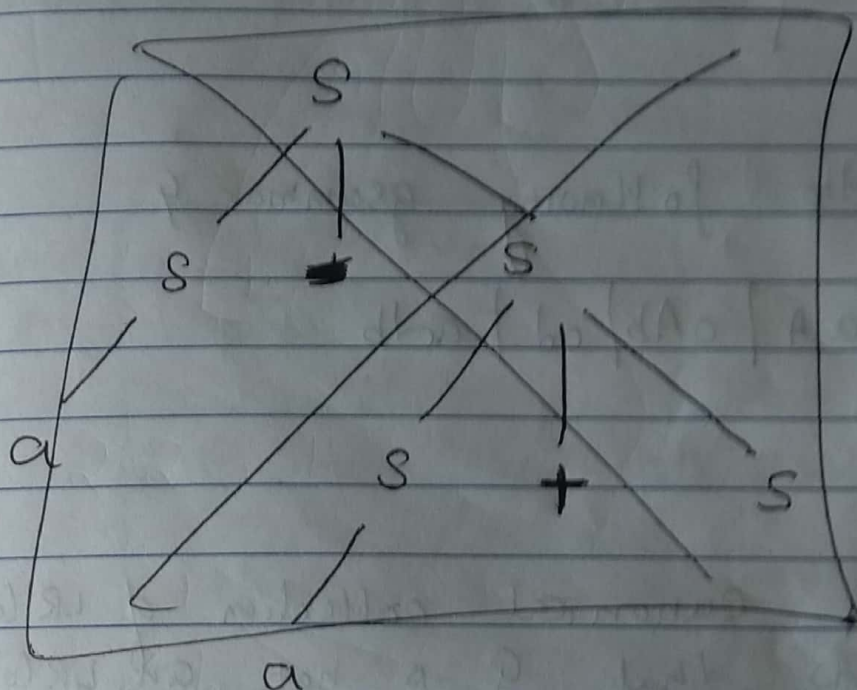
3 2 2 3 1

b

$$S \rightarrow S + S \quad -1$$
$$S \rightarrow S-S \quad -2$$
$$S \rightarrow (S) \quad -3$$
$$8 \rightarrow a - 4$$

i/p $a_1 - (a_2 + a_3)$

o/p ~~40-44~~ 44-132



Consider the following grammar G

$$S \rightarrow aA \mid cAb \mid cd \mid adb$$

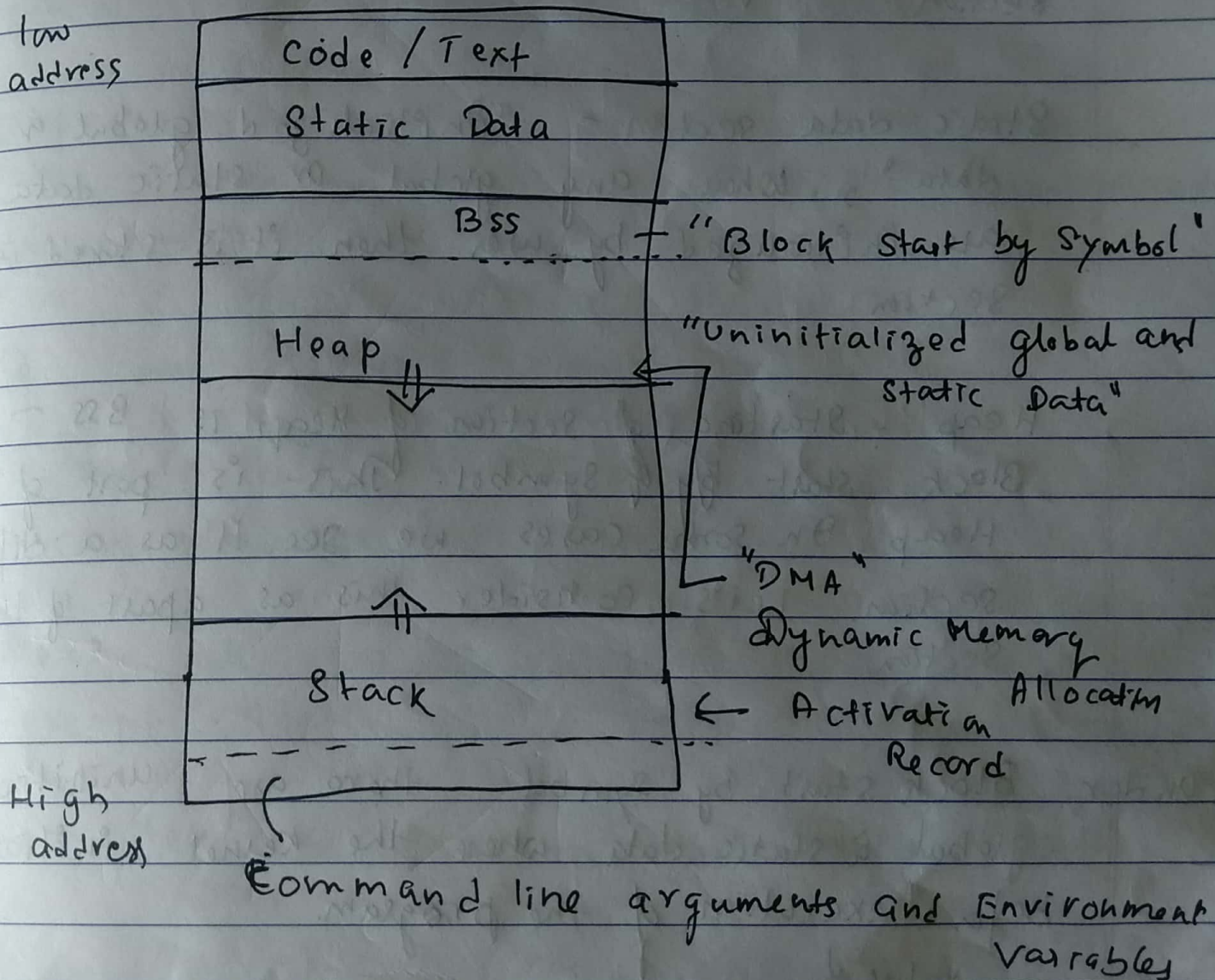
$$A \rightarrow d$$

- (i) Construct canonical collection of LR(0) items and show that G is not an LR(0) grammar.
- (ii) Compute Follow sets of the non-terminals and show that G is not an SLR(1) grammar. (You need not construct the complete SLR parser table. Just highlight all the states/s with conflict & justify)
- (iii) Construct the LALR(1) parser table for G .
- (iv) Construct canonical collection of LR(1) items and justify that G is an LALR(1) grammar.
- (v) Use the LALR parser table, parse the following strings
 - (a) cd^2b
 - (b) ad

Run-time Environment of a C-Program

When a program is under a execution then we say it as a process.

Consider the process structure



A run time process structure has code/text section, static data section, Heap and Stack

Code/Text - This is a read only section where you find a executable code

It is read only because the instructions do not get modified at any point of time

Thus we say this as code section or Text data section

Static data section - "Initialized global & static data", where any global or static data that are initialized by user then it is stored in this section

Heap - Starting of section of Heap is BSS - Block start by symbol. This is part of heap. In some cases we see it as a different section. Let's consider this as a part of the section.

Under Block start by symbol - there are uninitialized global & static data where the kernel initializes before execution of the program.

Thus, ^{value of} extern and static value by default is zero.

Because when this is utilized, the kernel would have already initialized external static value from 0 to 1.

The next section in heap is DMA which is "Dynamic Memory Allocation".

Under this we have malloc, calloc &.

Next is Stack, the bottom most of stack has command line arguments and environment variables.

In command line argument we have

`main (int argc, char *argv)`

and other envt variables.

Next, all the variables in the function except static will be there in the stack.

We call the stack an Activation Record.

Run time Environment

If we take any program C Program, Java programs then the program is stored in the hard disk. During compilation also the program resides in hard disk.

But ~~during~~ ~~only~~ a CPU or processor executes a program only when the program resides in main memory.

If the program resides in main memory then only the processor executor can execute the program.

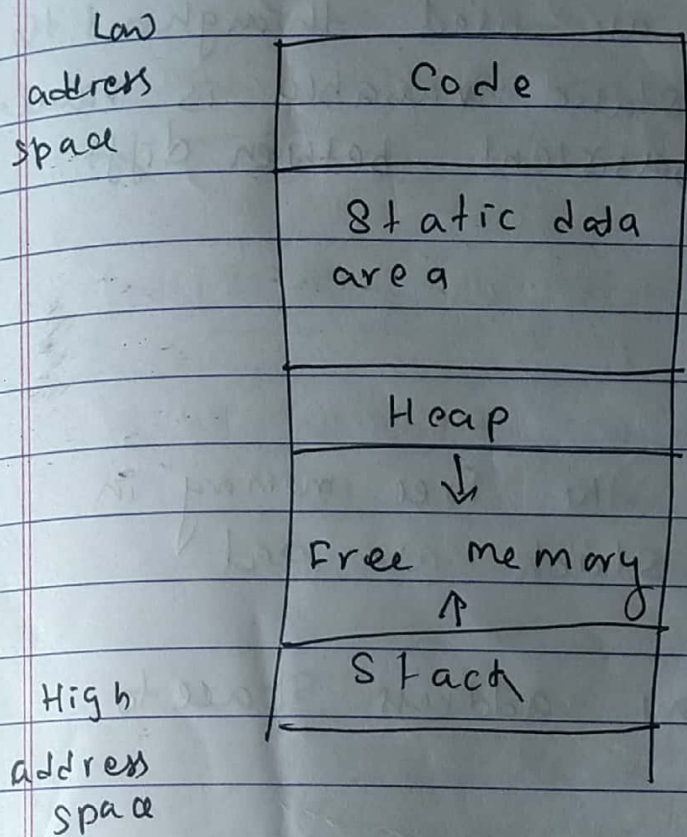
Block of program is compiled by the compiler. Now after compilation is over the compiler takes a block of memory from the operating system in order to store it in the main memory.

So, OS allocates a free block of memory to the corresponding program.

Compiler uses the free block of memory in order to store the compiled program.

So this is called Run time Storage Management.

Here we can divide the main memory into 4 parts



During the compilation the size of the program will be decided.

This code area mainly stores the executable code.

After compilation there is linking, linking means it links object code of several library files into a single file.

The code portion mainly contains .exe (that is nothing but the executable code)

Static data area - it is used to store the static variables & global variables.

[The global variables are used throughout the program whereas static variables is valued the variable is persistent between different function calls]

Heap & Stack

In order to utilize the free memory in effective heap & stack are used

Stack grows from low address space to high address space

Whereas heap grows from high address space to low address space.

~~Advantages~~

Stack is LIFO, where in order to store activation record, stack is used

So whenever a function or procedure call occurs then activation record ~~also~~ gets created

Model of Activation Record

or

Fields of Activation Record

Actual Parameters
Returned values
Control or dynamic link
Access or static link
Saved Machine Status
Local variables
Temporary variables

① Actual Parameters :- It holds the actual parameters of the calling function.

actual parameters the parameters declared inside the calling function

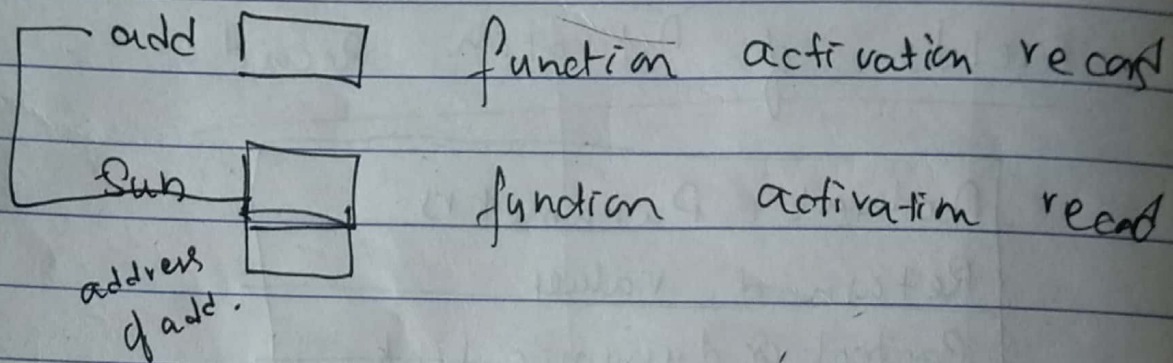
Eg: `func1(12, 23)` here 12 & 23 are actual parameters

② Returned values :- It is used to store the result of function call.

Eg:- `a = Sum();` - function call
`int Sum()` - function defn
3

Control link :- It points to the Activation record of calling function

```
add()      - calling function
{
    sub();  - called function
}
```



Access link :- It refers to the local data of called function but found in another Activation record.

```
int g = 12;
```

```
void A()
```

```
{
```

```
    printf("%d", g);
```

```
}
```

```
void main()
```

```
{
```

```
    A();
```

```
}
```

Saved Machine Status :-

Stores address of next instruction to be executed

Local variable These variables are local to a function

Temporary variable :- Needed during expression evaluation

Heap - In C lang,

malloc

calloc

realloc

& free function by using these 4 functions we can allocate & deallocate the memory at run-time.

If there are too many functions stack is used
If there are too many dynamic or pointer variables then heap is used.

Stack is ~~low~~ - Stored at high addresses
Heap is stored at low addresses.

Storage organization

or

Storage Allocation Strategies

There are 3 types of ~~storage~~ of storage allocation available:

1. Static allocation - static memory allocation
 2. Stack allocation
 3. Heap allocation
- } Dynamic Allocation

1. Static Allocation or Static Memory Allocation

The allocation of memory during compilation time is known as static ^{memory} allocation, for ordinary variables or for arrays.

Drawbacks of static Memory Allocation

① We must know in advance the size of the array.

Let us suppose the array size is $a[5]$ for 6 elements and perform the operations of 10 elements then the size of array is $a[10]$

② If more memory is allocated then the memory will be wasted.

Let us suppose the memory is allocated for 100 elements, whereas we use it for 5 elements

ie, $a[100]$

↓ only
 $a[5]$ is used

That means $a[95]$ - 95 elements is wasted.

- ③ If less memory is allocated than required then it causes the problems because it is not possible to perform the corresponding operations.

Let us assume that memory is allocated for 5 elements whereas it performs operations on 10 elements

$a[5]$
memory

$a[10]$
operations performed

Here we can perform operations for 1st 5 elements only not for the next 5 elements
(6-10)

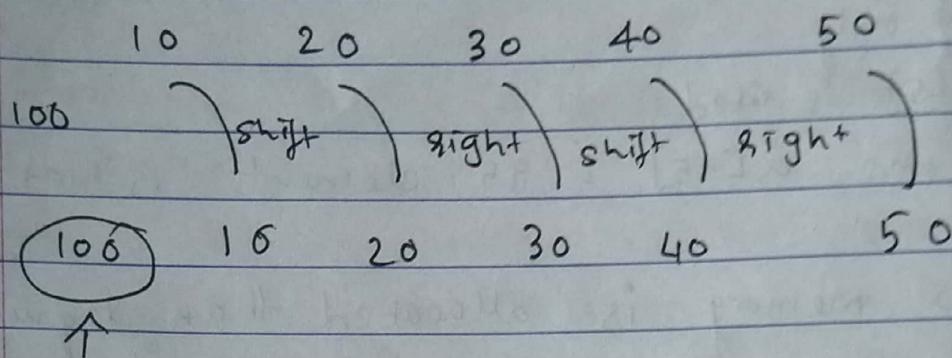
- ④ Insertion & deletion operations are very very expensive.

Consider elements :

10 20 30 40 50

I want to add element 100
=

Then for that I need to move all the element one position to the right



to
accomodate
this

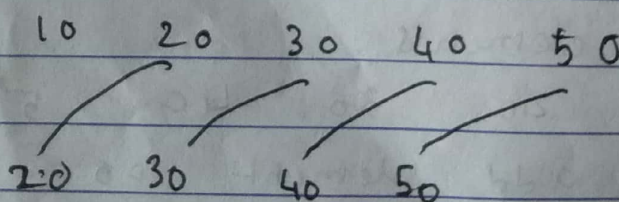
Let us assume the Array contains 1 lakh elements then 1 lakh shifting is required.

Like that deletion of the element is very very expensive.

Let us suppose,

10 20 30 40 50

and we want to delete first element (10)



So we will have to move each element to the left.

Same in this case if there are 1 lakh elements then 99,999 shiftings are needed.

So this is very very expensive

thus we use Dynamic Memory Allocation, which is nothing but memory allocation done during runtime. The memory will be allocated for the corresponding variables.

Dynamic Memory Allocation

① Stack allocation,

Here stack is a Data Structure which works in the form of LIFO (Last In First Out) i.e., last inserted can be deleted first.

Whenever a function or procedure call occurs then an activation record will be created for the corresponding function.

One activation record will be pushed up to the top of the stack.

Here we are using stack mainly for storing procedure or function information. That information resides in the activation record which is pushed on to the top of the stack.

If there are 5 functions in our ~~system~~ program then 5 activation records will be created for the corresponding 5 functions.

[What is activation record? Already explained in last class.]

Drawbacks of Stack Allocation

1. It supports dynamic memory allocation but slower than static allocation.
2. The stack allocation supports recursion but references to non local variables after activation records can't be retained.

In order to overcome this problem we used heap allocation.

Heap Allocation

So we mainly use Heap in order to ~~manage~~ increment Dynamic Memory Allocation.

If we use C++ then we can use dynamic memory allocation functions like: malloc, calloc, realloc, free functions.

By using those functions we can allocate the memory as well as we can change the size of the ^{already} allocated memory block.

and also release the memory block

On Java, we have new operator & delete operator. So by using those 2 functions we can allocate the memory as well as we can deallocate the memory.